



南京理工大学

3 分治策略

Divide and Conquer



引例：maxmin算法

求数组 $A[1...n]$ 的最小、最大元素问题

简单直观的方法：一个一个地找

先将 $A[1]$ 定义为min值和max值，

再在其余 $n-1$ 个元素 $A[i], i=2, \dots, n$, 中寻找，

若 $A[i] < \min$ 或 $A[i] > \max$ 则用 $A[i]$ 取代之。

复杂度：比较次数（基本运算）为 $2(n-1)$ 次。



maxmin算法用分治法如何求解？

当 $n=2$ 时，一次比较即可找到最小、最大值。

当 $n>2$ 时，可以把 n 个数据元素分为大致相等的两半，一半有 $\lfloor n/2 \rfloor$ 个数据元素，而另一半有 $\lceil n/2 \rceil$ 个数据元素。

先分别找出各自组中的最大元素和最小元素，然后

将两个最大元素进行比较，就可得 n 个元素的最大元素；
将两个最小元素进行比较，就可得 n 个元素的最小元素。



算法MINMAX

输入：n 个元素的数组 $A[1 \cdots n]$, n 为 2 的整数 k 次幂

输入： $A[1 \cdots n]$ 的最大最小元素

MINMAX(low, high)

1. if high-low=1
2. return (min($A[\text{low}]$, $A[\text{high}]$), max($A[\text{low}]$, $A[\text{high}]$))
3. else
4. mid = $\lfloor (\text{low} + \text{high}) / 2 \rfloor$
5. (x_1, y_1) $\leftarrow$ minmax(low, mid) {Devide}
6. (x_2, y_2) $\leftarrow$ minmax(mid+1, high) {Devide}
7. x $\leftarrow$ min(x_1, x_2) {Combine}
8. y $\leftarrow$ max(y_1, y_2) {Combine}
9. return (x, y)
10. end if



复杂度分析

元素比较次数 $C(n) = \begin{cases} 1 & \text{if } n = 2 \\ 2C(n/2) + 2 & \text{if } n > 2 \end{cases}$

$$C(n) = 2C(n/2) + 2$$

$$= 2\{2C(n/2^2) + 2\} + 2$$

$$= 2^2 C(n/2^2) + 2^2 + 2$$

$$= 2^2 \{2C(n/2^3) + 2\} + 4 + 2$$

$$= 2^3 C(n/2^3) + 2^3 + 2^2 + 2^1$$

$$= \dots$$

$$= 2^{k-1} C(n/2^{k-1}) + \{2^{k-1} + \dots + 2^2 + 2^1\} \quad // \text{ 设 } 2^k = n$$

$$= 2^{k-1} C(2) + \{2^k - 2\}$$

$$= 2^{k-1} + 2^k - 2$$

$$= 3n/2 - 2 \quad (\text{比简单直观的比较方法耗时少})$$



关于n的讨论

- 当 $n \neq 2^k$ 时，则n可表为若干2的正整数幂之和 (e.g. $42=32+8+2$), 若n是偶数，则算法的时间复杂度仍为 $3n/2-2$ 。
- e.g. 对 $n=42$ ，将元素分为32个一组、8个一组和2个一组共3组后， 各组找出最大、最小元素分别需要46、10和1次比较，
- 3个最大元素用2次比较即可得整体最大元素， 同理
- 3个最小元素用2次比较即可得整体最小元素，
- 故有 $46+10+1+2+2=61$,
- 而 $3n/2-2=3*42/2-2$ 也等于61， 故的确是 $3n/2-2$ 。



当 $n \neq 2^k$ 但为偶数时，可将 n 分解为若干2的正整数幂之和：

$$n = 2^{m_1} + 2^{m_2} + \dots + 2^{m_k} \quad \text{且}$$

$$m_1 > m_2 > \dots > m_k \geq 1,$$

对规模为 $n_i = 2^{m_i}$ 的每组元素，找出最大、最小元素需要 $3n_i/2 - 2$ 次比较，故获得 k 组各自的最大、最小元的时间复杂度为

$$\sum_{i=1}^k (3n_i / 2 - 2) = \frac{3}{2} \sum_{i=1}^k n_i - 2k = 3n / 2 - 2k$$

然后 k 个最大元、 k 个最小元**分别各用 $k-1$ 次比较**就可求出整体的最大元和最小元。

故全部的比较次数仍为 $(3n/2 - 2k) + (2k - 2) = \mathbf{3n/2 - 2}$ 。



当 $n \neq 2^k$ 但为奇数时, 则可将 n 分解为
若干个2的正整数幂之和加1, 即

$$n = 2^{m_1} + 2^{m_2} + \dots + 2^{m_k} + 1 \quad \text{且}$$

$$m_1 > m_2 > \dots > m_k \geq 1,$$

对规模为 $n_i = 2^{m_i}$ ($i=1, \dots, k$) 的每组元素, 找出最大、最小元素
需要 $3n_i/2-2$ 次比较, 故分别获得 k 组各自的最大、最小元的时间

复杂度为 $\sum_{i=1}^k (3n_i / 2 - 2) = \frac{3}{2} \sum_{i=1}^k n_i - 2k = \frac{3}{2}(n-1) - 2k$

注意此时共有 $k+1$ 组最大、最小元 (单独的一个元素, 既是该组
最大元, 也是该组最小元), 得到整体解还需比较 $2(k+1)-2$ 即 $2k$
次。故全部的比较次数为 $3(n-1)/2-2k + 2k = 3(n-1)/2$ 即 $3n/2-3/2$,
亦可看成为 $3n/2-2$ 。



分治策略的思想

- 把规模较大的问题分解为若干个规模较小的子问题，这些子问题相互独立且与原问题同类；（该子问题的规模减小到一定的程度就可以容易地解决）
- 依次求出这些子问题的解，然后把这些子问题的解组合起来得到原问题的解。
- 由于子问题与原问题是同类的,故分治法可以很自然地应用递归。



分治范式

- (a)划分Divide:
 - 将输入分成 $p \geq 1$ 个部分，每个子实例的规模严格小于 n ;
- (b)治理Conquer:
 - 如果问题的规模大于某个预定的阈值 n_0 ，这个步骤由执行 p 个递归调用组成。
- (c)组合Merge:
 - 组合 p 个递归调用的解来得到期望的输出值。



使用分治策略的算法设计模式

Divide-and-Conquer(P)

```
{  
  if ( $|P| \leq n_0$ ) direct_process(P);  
  divide P into smaller subinstances  
   $P_1, P_2, \dots, P_k$ ;  
  for( $i=1; i \leq k; i++$ )  
     $y_i = \text{Divide-and-Conquer}(P_i)$ ;  
  return Merge( $y_1, \dots, y_k$ );  
}
```

判断输入规模 p 是否足够的小

解该规模问题的函数

分割函数

分治解决

合并得到最后结果



使用分治策略的算法的时间复杂度分析

- 从分治法的一般设计模式可以看出，用它设计出的算法通常可以是递归算法。因而，算法的效率通常可以用递归方程来分析。
- 假设算法将规模为 n 的问题分解为 $a(a \geq 1)$ 个规模为 $n/b(b > 1)$ 的子问题解决。分解子问题以及合并子问题的解耗费的时间为 $s(n)$ ，则算法的时间复杂度可以递归表示为：

$$T(n) = \begin{cases} c & , n \leq n_0 \\ aT(n/b) + s(n) & , n > n_0 \end{cases}$$

- 回顾Master Theorem



Master Theorem

设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为一给定的函数, $T(n)$ 递归定义如下:

$$T(n) = a \cdot T(n/b) + f(n)$$

并且 $T(n)$ 有适当的初始值。那么, 当 n 充分大时, 有:

1) 若有 $\varepsilon > 0$, 使 $f(n) = O(n^{(\log_b a) - \varepsilon})$

(即 $f(n)$ 的量级多项式地小于 $n^{\log_b a}$ 的量级), 则 $T(n) = \Theta(n^{\log_b a})$ 。

2) 若 $f(n) = \Theta(n^{\log_b a})$

(即 $f(n)$ 的量级等于 $n^{\log_b a}$ 的量级), 则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

3) 若有 $\varepsilon > 0$, 使 $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$

(即 $f(n)$ 的量级多项式地大于 $n^{\log_b a}$ 的量级), 且满足正规性条件:

存在常数 $c < 1$, 使得对所有足够大的 n , 有 $a \cdot f(n/b) \leq c \cdot f(n)$,

则 $T(n) = \Theta(f(n))$ 。

正规性条件的直观含义:

对所有足够大的 n , a 个子问题的分解与再组合所需要的时间总和, 严格小于原问题的分解和组合所需要的时间。



Master Theorem

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

$$T(n) = aT(n/b) + f(n)$$

$$= a\{aT(n/b^2) + f(n/b)\} + f(n)$$

$$= a^2T(n/b^2) + af(n/b) + f(n) \quad // \quad a \cdot f(n/b) \leq c \cdot f(n)$$

$$= a^2\{aT(n/b^3) + f(n/b^2)\} + af(n/b) + f(n)$$

$$= a^3T(n/b^3) + a^2f(n/b^2) + af(n/b) + f(n)$$

$$= \dots // a^2f(n/b^2) = a\{af(n/b^2)\} \leq a\{c \cdot f(n/b)\} = c\{a \cdot f(n/b)\} \leq c^2 \cdot f(n) < c \cdot f(n)$$

$$= a^kT(n/b^k) + \sum a^j f(n/b^j) \quad \{j=0, \dots, k-1\} \quad // \text{设 } b^k = n, \text{ 则 } k = \log_b n$$

$$\leq a^{\log_b n} + f(n) \cdot \sum c^j \quad \{j=0, \dots, k-1\}$$

$$< n^{\log_b a} + f(n) \cdot (1/(1-c))$$



快速排序



快速排序是一个非常流行而且高效的算法，其平均时间复杂度为 $\Theta(n \log n)$ 。其优于合并排序之处在于它是在原位置上排序，不需要额外的辅助存储空间(合并排序需 $\Theta(n)$ 的辅助空间)。**Charles A. R. Hoare** 1960年发布了使他闻名于世的快速排序算法(**Quicksort**)，这个算法也是当前世界上使用最广泛的算法之一，当时他供职于伦敦一家不大的计算机生产厂家。1980年，**Hoare**被授予**Turing**奖，以表彰其在程序语言定义与设计领域的根本性的贡献。在2000年，**Hoare**因其在计算机科学和教育方面的杰出贡献被英国皇家封为爵士。



将 $A[1...8]=\{4, 6, 3, 1, 8, 7, 2, 5\}$ 按照非降序方式进行排序

- **SPLIT**:以待排序数组的首元素作为基准元素，将待排序数组分成左右两个子数组。使得左边子数组中的元素都小于等于基准元素；右边子数组中的元素都大于等于基准元素。
- 对左、右子数组进行相同的操作。

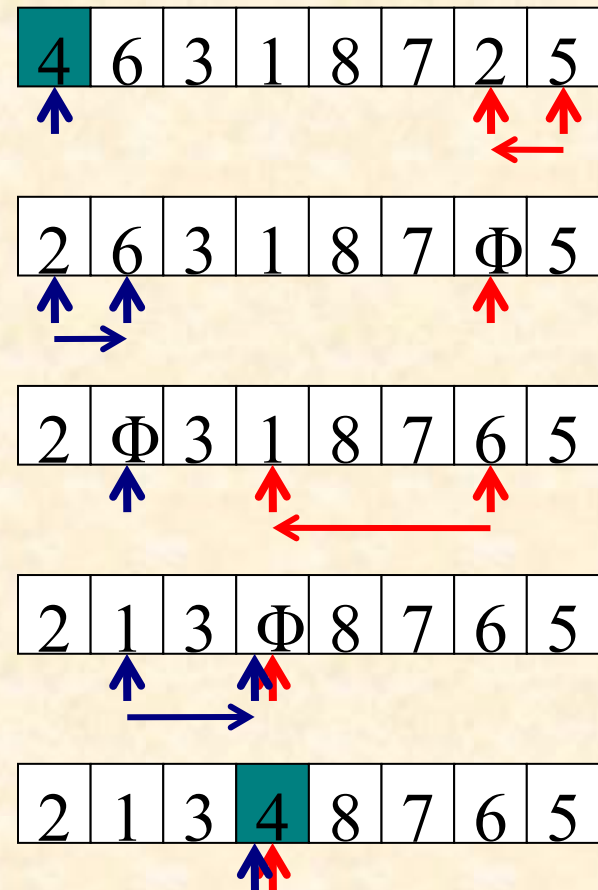


Algorithm: SPLIT(A[low,...high])

输入：数组A[low,...high]

输出：用A[low]作基准元素划分后的数组A
及基准元素新的位置w

1. $x \leftarrow A[\text{low}]$
2. while (low < high)
3. while (low < high && A[high] > x) --high;
4. A[low] $\leftarrow$ A[high]
5. while (low < high && A[low] $\leq$ x) ++low;
6. A[high] $\leftarrow$ A[low]
7. end while
8. A[low] $\leftarrow$ x
9. w $\leftarrow$ low
10. return A and w //新数组A与x的新位置w





Algorithm: QUICKSORT($A[\text{low} \dots \text{high}]$)

输入: n 个元素的数组 $A[\text{low} \dots \text{high}]$

输出: 按非降序排列的数组 $A[\text{low} \dots \text{high}]$

1. if $\text{low} < \text{high}$ then
2. $w \leftarrow \text{SPLIT}(A[\text{low} \dots \text{high}])$ { w 为基准元素 $A[\text{low}]$ 的新位置}
3. quicksort($A, \text{low}, w-1$)
4. quicksort($A, w+1, \text{high}$)
5. end if

主函数调用: QUICKSORT($A[1, n]$)



南京理工大学

4	6	3	1	8	7	2	5
2	1	3	4	8	7	6	5

2	1	3
1	2	3

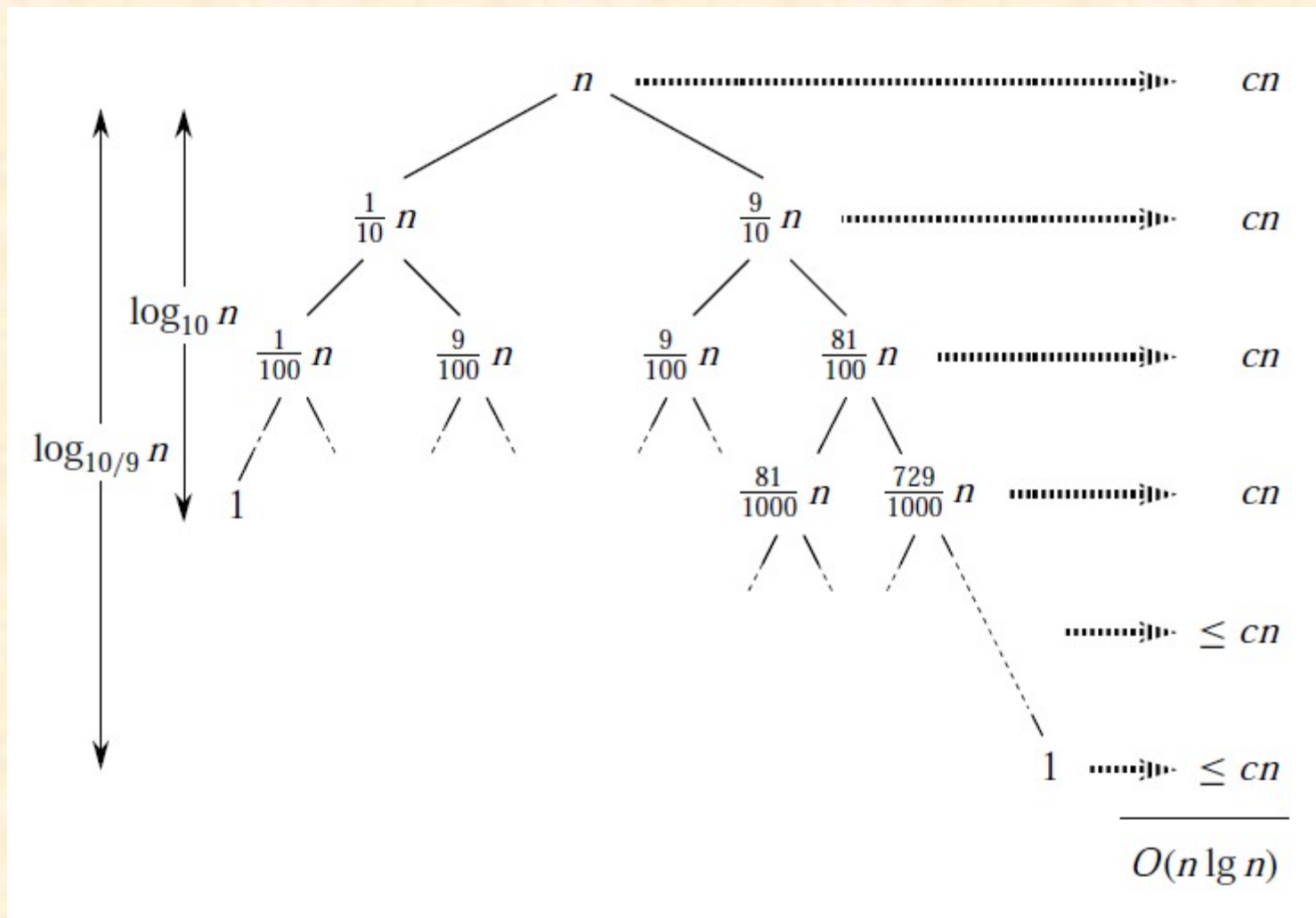
1
1

3
3

8	7	6	5
5	7	6	8

5	7	6
5	7	6

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---





时间复杂度分析

理想情形：每次SPLIT后得到的左右子数组规模相当，因此有：

$$T(n) = \begin{cases} O(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases} \implies T(n) = \Theta(n \log n)$$

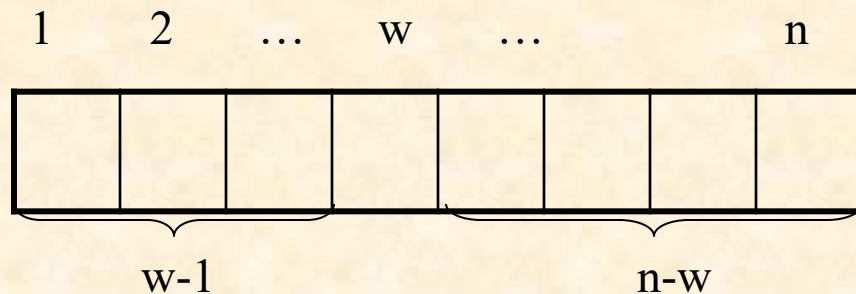
最差情形(已经排好序或是逆序的数组)：每次SPLIT后，只得到左或是右子数组，因此有：

$$T(n) = \begin{cases} O(1) & \text{if } n = 1 \\ T(n-1) + \Theta(n) & \text{if } n > 1 \end{cases} \implies T(n) = \Theta(n^2)$$



平均情形：

我们用 $C(n)$ 表示对一个 n 个元素的数组进行快速排序所需要的总的比较次数。



因此，我们有：

$$C(n) = (n-1) + \frac{1}{n} \sum_{w=1}^n (C(w-1) + C(n-w))$$

$$\because \sum_{w=1}^n C(n-w) = C(n-1) + C(n-2) + \cdots + C(0) = \sum_{w=1}^n C(w-1)$$

$$\therefore C(n) = (n-1) + \frac{2}{n} \sum_{w=1}^n C(w-1)$$



$$n \cdot C(n) = n(n-1) + 2 \sum_{w=1}^n C(w-1) \dots (a)$$

↓ n-1 替换 n

$$(n-1)C(n-1) = (n-1)(n-2) + 2 \sum_{w=1}^{n-1} C(w-1) \dots (b)$$

(a)-(b), 并适当变换

→
$$\frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

令 $D(n) = \frac{C(n)}{n+1}$ ↓

$$D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}, D(1) = 0$$

↓

$$D(n) = 2 \sum_{j=1}^n \frac{j-1}{j(j+1)} = 2 \sum_{j=1}^n \frac{2}{(j+1)} - 2 \sum_{j=1}^n \frac{1}{j}$$

$$= 4 \sum_{j=2}^{n+1} \frac{1}{j} - 2 \sum_{j=1}^n \frac{1}{j} = 2 \sum_{j=1}^n \frac{1}{j} - \frac{4n}{n+1}$$



南京理工大学

$$\begin{aligned} D(n) &= 2 \sum_{j=1}^n \frac{1}{j} - \frac{4n}{n+1} \\ &= 2 \ln n - \frac{4n}{n+1} + o(1) \end{aligned}$$

$$\frac{1}{n} + \ln n \leq \sum_{j=1}^n \frac{1}{j} \leq 1 + \ln n$$

$$\begin{aligned} C(n) &= (n+1)D(n) \\ &= (n+1) \left\{ 2 \ln n - \frac{4n}{n+1} + o(1) \right\} \\ &= 2(n+1) \frac{\log n}{\log e} - 4n + (n+1)o(1) \\ &= \frac{2}{\log e} n \log n - 4n + \frac{2}{\log e} \log n + (n+1)o(1) \\ &= \Theta(n \log n) \end{aligned}$$



合并排序Merge Sort

例：给定数组A [1...8]=

8	4	3	1	6	2	9	7
---	---	---	---	---	---	---	---

1. 将其分分成左右两个子数组:

8	4	3	1
---	---	---	---

6	2	9	7
---	---	---	---

2. 对子数组进行排序(可采用任何排序方法)。

3. 对排序后的子数组进行合并 :两个已排序的子数组用A[p...q]和A[q+1...r]表示.
设两个指针s和t, 初始时各自指向A[p]和A[q+1],再设一空数组B[p...q, q+1...r]
做暂存器, 比较元素A[s]和A[t], 将较小者添加到B, 然后移动指针,
若A[s]较小, 则s+1, 否则t+1,
直到s=q+1 或 t=r+1 为止
将剩余元素A[t...r] 或 A[s...q] 拷贝到数组B, 然后令A←B.

例:

2	3	6
---	---	---

7	11	13	45	57
---	----	----	----	----



Algorithm: MERGE(A, p, q, r)

输入：数组A[p...q]和A[q+1...r], 各自按升序排列

输出：将A[p...q]和A[q+1...r]合并成一个升序排序的新数组

1. $i \leftarrow p; j \leftarrow q+1; k \leftarrow p;$
2. **while** $i \leq q$ **and** $j \leq r$
3. if $A[i] \leq A[j]$ then
4. $B[k] \leftarrow A[i]$
5. $i \leftarrow i+1$
6. else
7. $B[k] \leftarrow A[j]$
8. $j \leftarrow j+1$
9. end if
10. $k \leftarrow k+1$
11. **end while**
12. **if** $i=q+1$ **then** $B[k...r] \leftarrow A[j...r]$
13. **else** $B[k...r] \leftarrow A[i...q]$
14. **end if**
15. $A \leftarrow B$



Algorithm: MERGESORT(A, low, high)

输入：待排序数组A[low...high]

输出：A[low...high]按非降序排列

1. if low < high then

2. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$

3. MERGESORT(A, low, **mid**)

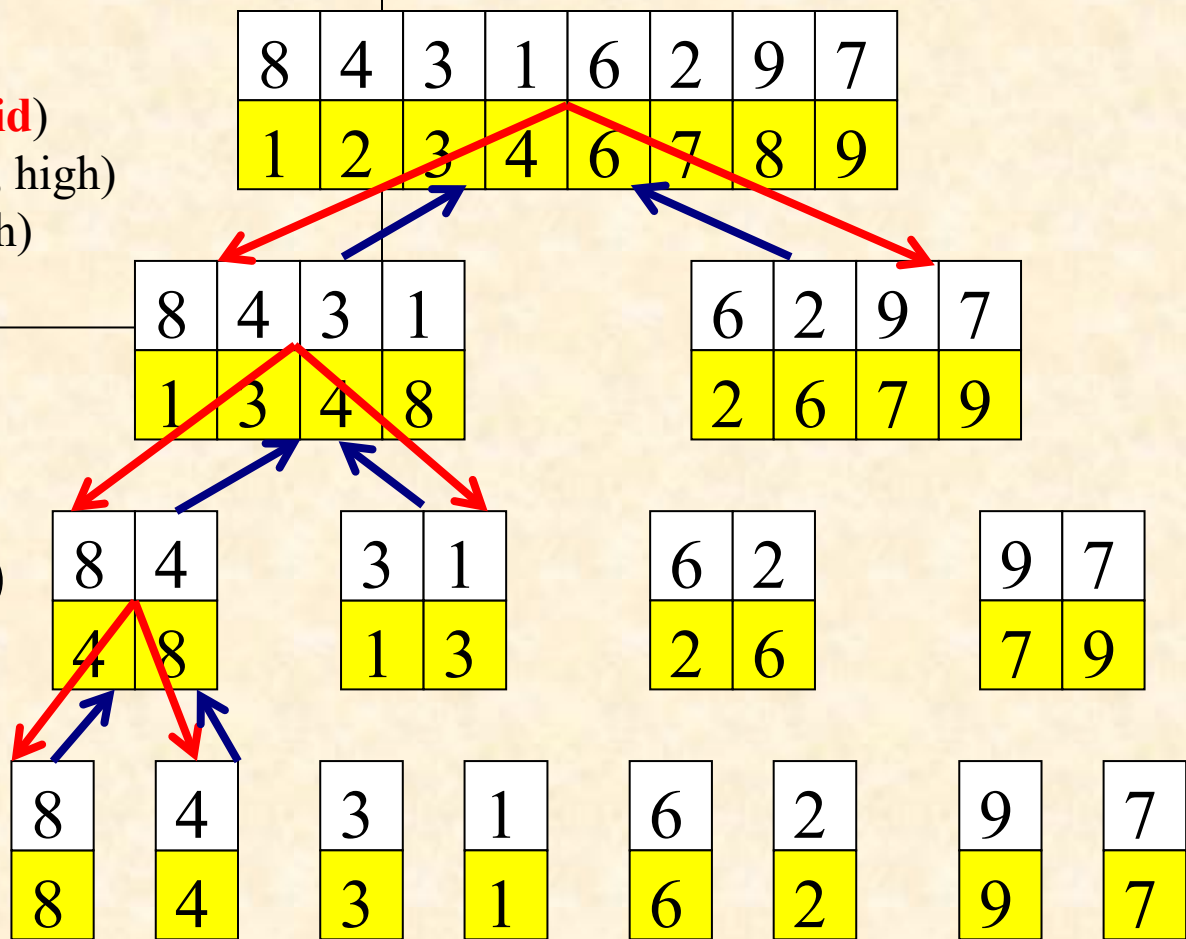
4. MERGESORT(A, **mid+1**, high)

5. MERGE(A, low, mid, high)

6. end if

主函数调用：

mergesort (A, 1, n)





时间复杂度分析

最小比较次数

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2C(n/2) + n/2 & \text{if } n \geq 2 \end{cases} \quad \longrightarrow \quad C(n) = \frac{n \log n}{2}$$

最大比较次数

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2C(n/2) + n - 1 & \text{if } n \geq 2 \end{cases} \quad \longrightarrow \quad C(n) = n \log n - n + 1$$



- 若 n 是任意的正整数，则：

$$C(n) = \begin{cases} 0 & \text{若 } n=1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + bn & \text{若 } n \geq 2 \end{cases}$$

$$C(n) = \Theta(n \log n) \quad \text{参考定理2.6, pp.60}$$

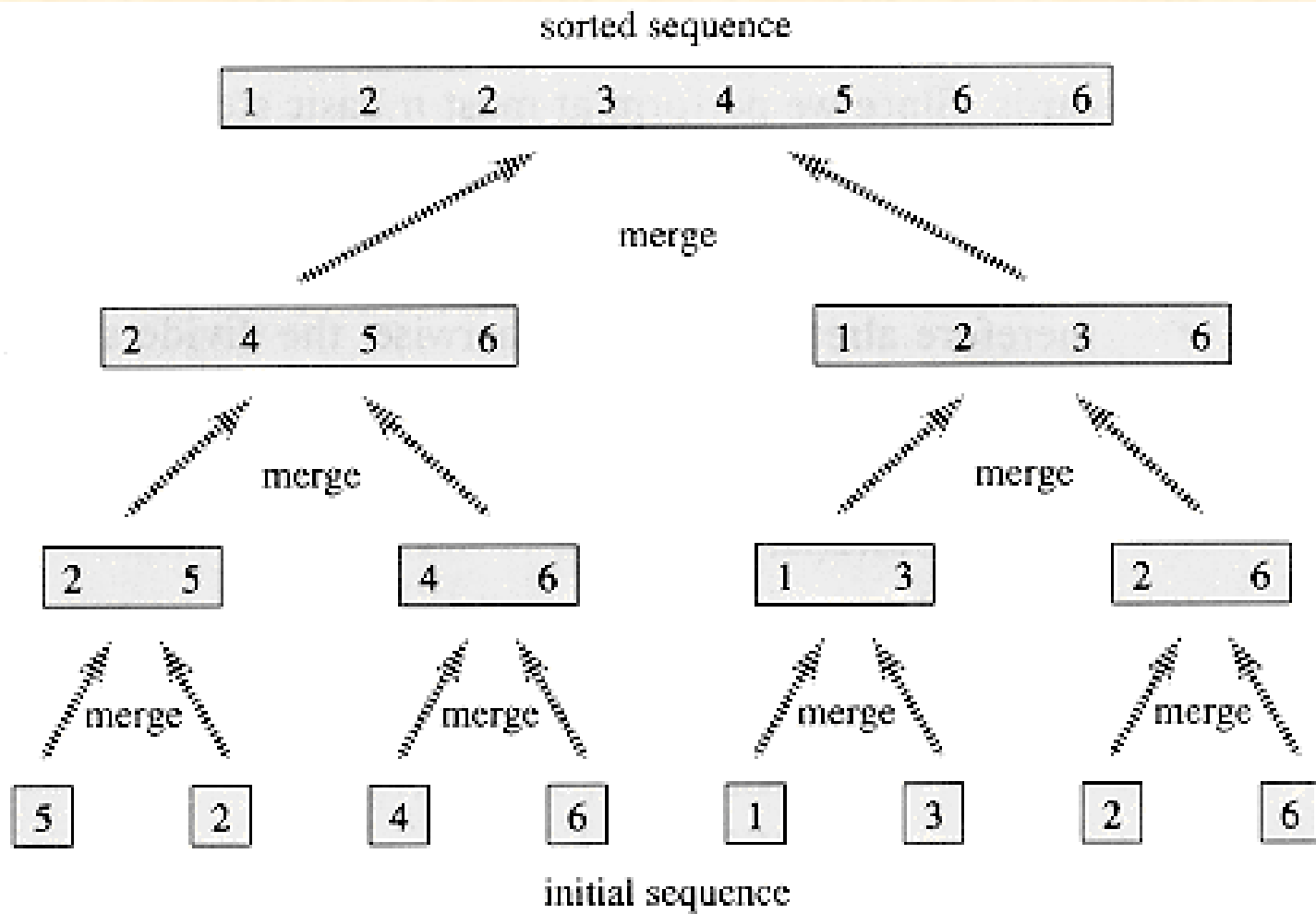


定理 6.3

算法**MERGESORT**对一个 **n** 个元素的数组排序所需的时间是 $\Theta(n \log n)$ ，空间是 $\Theta(n)$ 。



BottomUp Merge





Consider?

- 既然有BOTTOMUPSORT迭代算法，为什么还要借助于像MERGESORT那样的递归算法？特别是考虑到因使用栈而需要的额外空间数，以及由处理递归调用内在开销带来的额外空间。

递归算法具有对问题易于叙述、领会和分析等优点，算法的设计者可以从对递归描述的框架着手，如果可能，以后再将算法改进并转化为一个迭代算法。



排序算法：复杂度对比

算法	平均时间	最坏时间	辅助空间
简单排序 (插入、冒泡、选择)	$O(n^2)$	$O(n^2)$	$O(1)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$
(自下而上) 归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$



矩阵乘法

- 设 A, B 是两个 $n \times n$ 的矩阵，求 $C=AB$.
- 方法1: 直接相乘法
- 方法2: 分块矩阵法(直接应用分治策略)
- 方法3: Strassen算法(改进的分治策略)



方法1: 直接相乘

$$C = [c_{ij}]_{i=1,2,\dots,n; j=1,2,\dots,n} \quad c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

时间复杂度分析：假设每做一次标量乘法耗费时间为 m ,每做一次标量加法耗费时间为 a ,那么直接相乘算法的时间复杂度为：

$$T(n) = n^3 m + n^2 (n-1) a = \Theta(n^3)$$



方法2:分块矩阵法(直接应用分治策略)

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} \mathbf{C}_{11} & \mathbf{C}_{12} \\ \mathbf{C}_{21} & \mathbf{C}_{22} \end{bmatrix}$$

$$\mathbf{C}_{11} = \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21}$$

$$\mathbf{C}_{12} = \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22}$$

$$\mathbf{C}_{21} = \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21}$$

$$\mathbf{C}_{22} = \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22}$$

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 8T(n/2) + 4(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$



$$T(n) = \Theta(n^3)$$



Strassen算法



Volker Strassen
giving the Knuth Prize
lecture at SODA 2009.

Strassen was born on April 29, 1936, in Germany. In 1969, Strassen shifted his research efforts towards the analysis of algorithms with a paper on Gaussian elimination, introducing Strassen's algorithm, **the first algorithm** for performing **matrix multiplication faster than** the $O(n^3)$ time bound that would result from a naive algorithm. In the same paper he also presented an asymptotically-fast algorithm to perform matrix inversion, based on the fast matrix multiplication algorithm. **This result was an important theoretical breakthrough**, leading to much additional research on fast matrix multiplication, and despite later theoretical improvements it remains a practical method for multiplication of dense matrices of moderate to large sizes.

—From Wikipedia, the free encyclopedia



引入下列 $M_i(i=1,2,\cdots,7)$:

$$M_1=(A_{12}-A_{22})(B_{21}+B_{22})$$

$$M_2=(A_{11}+A_{22})(B_{11}+B_{12})$$

$$M_3=(A_{11}-A_{21})(B_{11}+B_{12})$$

$$M_4=(A_{11}+A_{12})B_{22}$$

$$M_5=A_{11}(B_{12}-B_{22})$$

$$M_6=A_{22}(B_{21}-B_{11})$$

$$M_7=(A_{21}+A_{22})B_{11}$$

则有: $C_{11}=M_1+M_2-M_4+M_6$, $C_{12}=M_4+M_5$,

$$C_{21}=M_6+M_7,$$

$$C_{22}=M_2-M_3+M_5-M_7$$

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 7T(n/2) + 18(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$



$$T(n) = \Theta(n^{\log_b^a}) = \Theta(n^{\log_2^7}) = \Theta(n^{2.81})$$

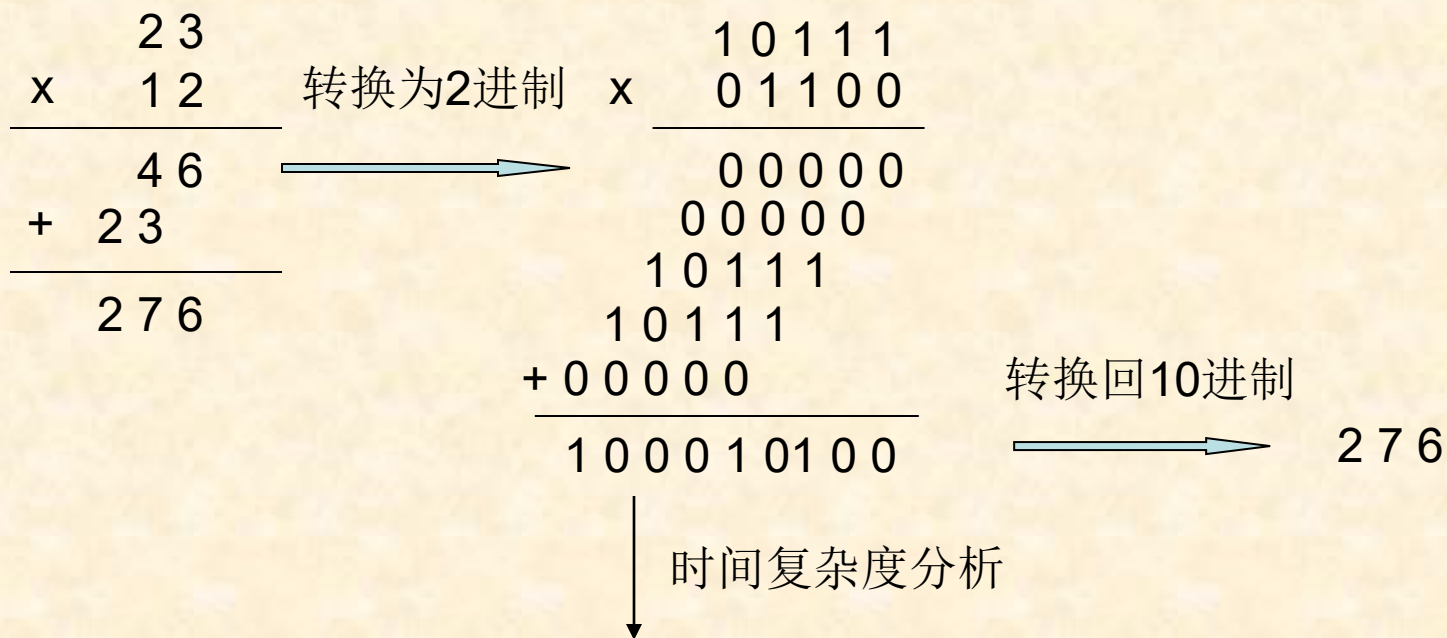


大整数相乘

- 通常，在分析算法的计算复杂度时，都将加法和乘法运算当作基本运算来处理，即将执行一次加法或乘法运算所需要的计算时间当作一个常数，该常数仅仅取决于计算机硬件处理速度。
- 然而，这个假定仅仅在参加运算的整数处于一定范围内时才是合理的。这个整数范围取决于计算机硬件对整数的表示。
- 在某些情况下，要处理很大的整数，它无法在计算机硬件能直接表示的整数范围内进行处理。这时候，就必须使用软件的方法来实现大整数的算术运算。



- 问题：设有两个n bit位的二进制整数X, Y，要计算XY.



将位(bit)的乘法或加法当作基本运算，则逐位相乘算法的时间复杂度为：

$$T(n) = \Theta(n^2)$$



使用分治策略来求解之

$X = \begin{array}{|c|c|} \hline \frac{n}{2} & \frac{n}{2} \\ \hline A & B \\ \hline \end{array} \quad Y = \begin{array}{|c|c|} \hline \frac{n}{2} & \frac{n}{2} \\ \hline C & D \\ \hline \end{array} \xrightarrow{\text{例}} X = \begin{array}{|c|c|} \hline 1 & 1 \\ \hline 0 & 1 \\ \hline \end{array}$

我们有: $\downarrow$

$X = A \cdot 2^{n/2} + B \quad Y = C \cdot 2^{n/2} + D \xrightarrow{\text{例}} X = 3 \cdot 2^{4/2} + 1 = 13$

下式成立: $\downarrow$

$$\begin{aligned} XY &= (A \cdot 2^{n/2} + B)(C \cdot 2^{n/2} + D) \\ &= A \cdot C 2^n + (A \cdot D + C \cdot B) 2^{n/2} + B \cdot D \end{aligned}$$



时间复杂性分析

$$\begin{aligned} XY &= (A \cdot 2^{n/2} + B)(C \cdot 2^{n/2} + D) \\ &= A \cdot C \cdot 2^n + (A \cdot D + C \cdot B) \cdot 2^{n/2} + B \cdot D \end{aligned}$$

(1) 4次 $n/2$ bit位数的乘法($A \cdot C, A \cdot D, C \cdot B, B \cdot D$) // $4T(n/2)$

(2) $A \cdot C$ 左移 n 位 ($A \cdot C \cdot 2^n$)

(3) 求 $A \cdot D + B \cdot C$ 所做的 n 位加法

(4) $A \cdot D + B \cdot C$ 左移 $n/2$ 位 ($(A \cdot D + B \cdot C) \cdot 2^{n/2}$)

(5) 两个加法 ($A \cdot C + ((A \cdot D + B \cdot C) \cdot 2^{n/2} + B \cdot D)$)

$\Theta(n)$



$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 4T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases} \Longrightarrow T(n) = \Theta(n^2)$$

如何降低时间复杂性？

观察： $A \cdot D + B \cdot C = (A + B)(C + D) - AC - BD$

所以：

$$\begin{aligned} XY &= (A \cdot 2^{n/2} + B)(C \cdot 2^{n/2} + D) \\ &= A \cdot C \cdot 2^n + (A \cdot D + C \cdot B) \cdot 2^{n/2} + B \cdot D \\ &= \underline{A \cdot C} \cdot 2^n + ((\underline{A + B})(\underline{C + D}) - AC - BD) \cdot 2^{n/2} + \underline{B \cdot D} \end{aligned}$$

$$T(n) = \begin{cases} a & , \text{if } n = 1 \\ 3T(n/2) + \Theta(n) & , \text{if } n > 1 \end{cases} \Longrightarrow T(n) = \Theta(n^{\log 3}) = \Theta(n^{1.59})$$



$$XY = (A \cdot 2^{n/2} + B)(C \cdot 2^{n/2} + D)$$

进一步的思考

$$= A \cdot C \cdot 2^n + (A \cdot D + C \cdot B) \cdot 2^{n/2} + B \cdot D$$

1) 计算乘法 $(A+B)(C+D)$ 时， $(A+B)$ 或 $(C+D)$ 可能超过 $n/2$ 位，使问题规模变大。实际上， $(A \cdot D + B \cdot C)$ 与 $A \cdot C, B \cdot D$ 之间还存在如下关系：

$$(A \cdot D + B \cdot C) = (B - A)(C - D) + A \cdot C + B \cdot D$$

则 $XY = A \cdot C \cdot 2^n + (A \cdot D + B \cdot C) \cdot 2^{n/2} + B \cdot D$

$$= A \cdot C \cdot 2^n + \{(B - A)(C - D) + A \cdot C + B \cdot D\} \cdot 2^{n/2} + B \cdot D$$

2) 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。



二分搜索算法

输入：非降序排列的数组 $A[1\dots n]$ 和元素 x

输出：如果 $x=A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0 .

1. $low \leftarrow 1; high \leftarrow n; j \leftarrow 0$
2. while($low \leq high$) and ($j=0$)
3. $mid \leftarrow \lfloor (low+high)/2 \rfloor$
4. if $x=A[mid]$ then $j \leftarrow mid$
5. else if $x < A[mid]$ then $high \leftarrow mid-1$
6. else $low \leftarrow mid+1$
7. end while
8. return j



时间复杂度分析

- 最好情形：比较1次
- 最坏情形：比较 $\lfloor \log n \rfloor + 1$ 次
 - 每次循环都要抛弃一些元素，例如第二次循环时，剩余元素为 $A[1 \dots \text{mid}-1]$ 或 $A[\text{mid}+1 \dots n]$ ，不妨设为 $A[\text{mid}+1 \dots n]$ ，则剩余的元素个数是 $\lfloor n/2 \rfloor$
 - 第 j 次while循环时，剩余元素的个数是 $\lfloor n/2^{j-1} \rfloor$
 - 或者找到 x ，或者程序在子序列长度达到1时终止搜索，此时 $\lfloor n/2^{j-1} \rfloor = 1 \Leftrightarrow 1 \leq n/2^{j-1} < 2 \Leftrightarrow 2^{j-1} \leq n < 2^j \Leftrightarrow \log n < j \leq \log n + 1 \Leftrightarrow j = \lfloor \log n \rfloor + 1$



时间复杂度分析—递归方法

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\lfloor n/2 \rfloor) + 1 & \text{if } n \geq 2 \end{cases}$$

$$\Rightarrow T(n) = T(\lfloor n/2 \rfloor) + 1 = T(\lfloor n/2^2 \rfloor) + 1 + 1$$

$$= T(\lfloor n/2^{\lfloor \log n \rfloor} \rfloor) + \lfloor \log n \rfloor = 1 + \lfloor \log n \rfloor = \Theta(\log n)$$



平均时间复杂度分析

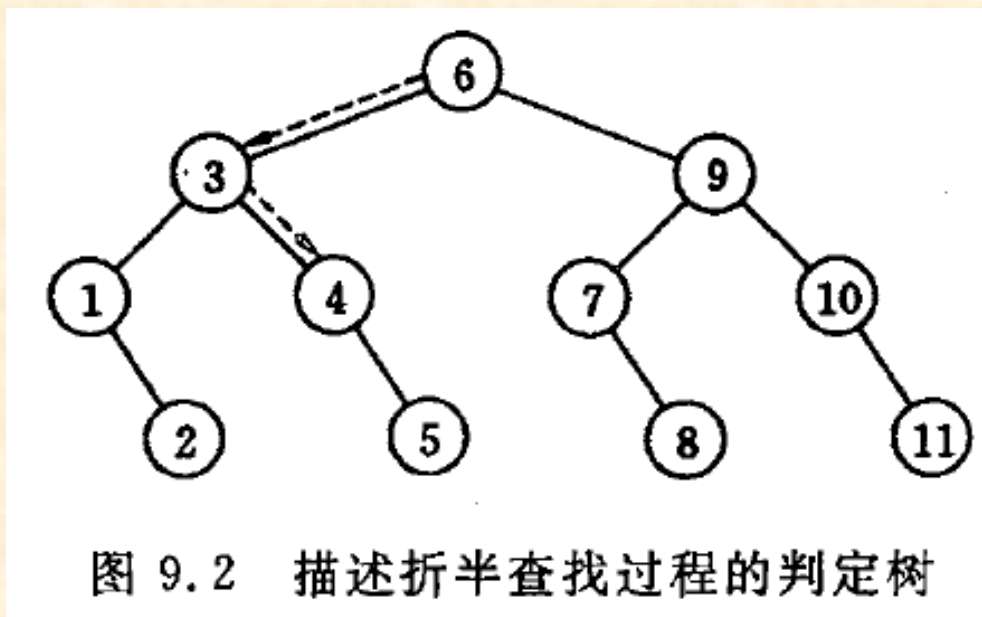


图 9.2 描述折半查找过程的判定树

假设：有序数组长度 $n=2^{h+1}-1$ ，则折半查找对应于一个高度为 h （设根高度为0）的完全二叉树，假设每个关键字的查找概率均为 $1/n$ ，则平均查找长度为：

$$ASL = \sum_{i=1}^n p_i L_i$$



平均时间复杂度分析

$$ASL = \sum_{i=1}^n p_i L_i$$

$$= \frac{1}{n} \sum_{j=0}^h (j+1) \times 2^j$$

假设为完全二叉树，树高为h，第j层均有 2^j 个结点，则有 $n=2^{h+1}-1$

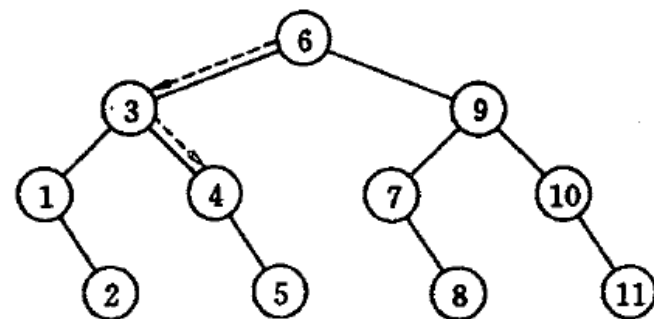


图 9.2 描述折半查找过程的判定树

$$\sum_{j=0}^h (j+1) \times 2^j = 1 \times 2^0 + 2 \times 2^1 + 3 \times 2^2 + \dots + h \times 2^{h-1} + (h+1) \times 2^h = S$$

$$2S = 1 \times 2^1 + 2 \times 2^2 + \dots + h \times 2^h + (h+1) \times 2^{h+1}$$

$$S = (h+1) \times 2^{h+1} - (2^0 + 2^1 + \dots + 2^h)$$

$$= (h+1) \times 2^{h+1} - (2^{h+1} - 1)$$

$$= h2^{h+1} + 1$$

$$ASL = \frac{1}{n} \sum_{j=0}^h (j+1) \times 2^j = \frac{1}{n} [h2^{h+1} + 1] \Big|_{h = \log(n+1) - 1} = \frac{n+1}{n} \log(n+1) - 1 \approx \log(n+1) - 1$$



平均时间复杂度分析

假设树是几乎完全的二叉树，有 n 个结点，树高为 h ，这里 $h = \lfloor \log n \rfloor$ ，第 j 层（前 $h-1$ 层）均有 2^j 个结点，第 j 层的结点则未必满。

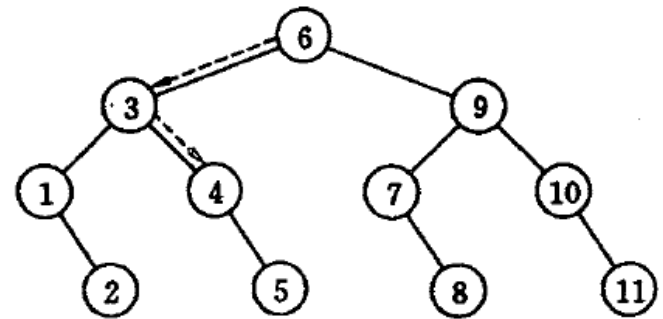


图 9.2 描述折半查找过程的判定树

$$ASL = \sum_{i=1}^n p_i L_i$$

$$= \frac{1}{n} \left\{ \sum_{j=0}^{h-1} (j+1) \times 2^j + \left(n - \sum_{j=0}^{h-1} 2^j \right) \times (h+1) \right\}$$

$$\sum_{j=0}^{h-1} (j+1) \times 2^j = 1 \times 2^0 + 2 \times 2^1 + 3 \times 2^2 + \dots + h \times 2^{h-1} = S$$

$$2S = 1 \times 2^1 + 2 \times 2^2 + \dots + (h-1) \times 2^{h-1} + h \times 2^h$$

$$S = h \times 2^h - (2^0 + 2^1 + \dots + 2^{h-1})$$

$$= h \times 2^h - (2^h - 1)$$



平均时间复杂度分析

$$\begin{aligned} ASL &= \sum_{i=1}^n p_i L_i \\ &= \frac{1}{n} \left\{ \sum_{j=0}^{h-1} (j+1) \times 2^j + \left(n - \sum_{j=0}^{h-1} 2^j \right) \times (h+1) \right\} \\ &= \frac{1}{n} \left\{ h \times 2^h - (2^h - 1) + [n - (2^h - 1)] \times (h+1) \right\} \\ &= \frac{1}{n} \left\{ (n+1)(h+1) + 1 - 2^{h+1} \right\} \Big|_{h = \lfloor \log_2 n \rfloor} \\ &= \lfloor \log_2 n \rfloor + 1 - \left\{ 2 \times \frac{2^{\lfloor \log_2 n \rfloor}}{n} - \frac{\lfloor \log_2 n \rfloor + 2}{n} \right\} \approx \lfloor \log_2 n \rfloor + 1 - 2 \end{aligned}$$

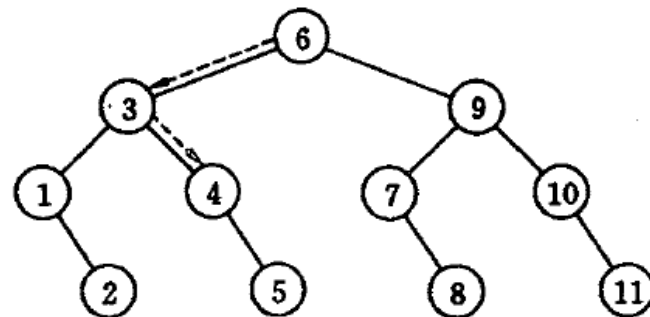


图 9.2 描述折半查找过程的判定树

二分查找的平均时间复杂度相当于：在最坏时间复杂度 $\lfloor \log n \rfloor + 1$ 的基础上，减去一个修正项，**修正项大小 ≈ 2**



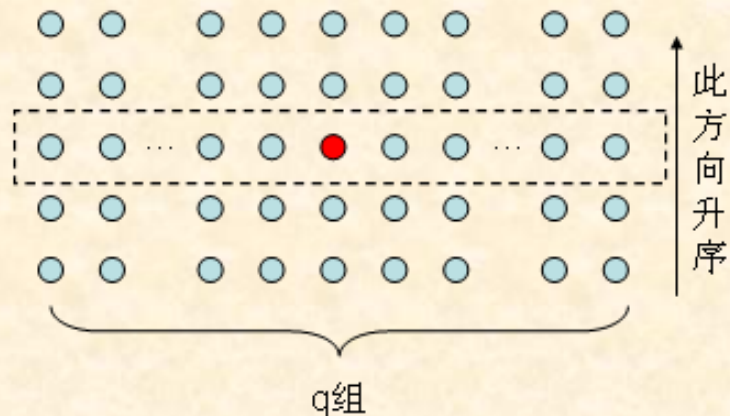
寻找中项和第k小元素

- 给定已排好序(非降序)的数组 $A[1...n]$,中项是指其“中间”元素。若 n 为奇数,则中项为数组中的第 $(n+1)/2$ 个元素;若 n 为偶数,则存在两个中间元素,分别为第 $n/2$ 和第 $n/2+1$ 个元素,在这种情形下,我们取第 $n/2$ 个元素作为中项;综上,中项为第 $\lceil n/2 \rceil$ 个最小元素。
- 寻找中项的一个直接的方法:先排序,后取中项。显然,该方法的时间复杂度至少为 $\Omega(n\log n)$ 。能够找到更为高效的方法?
- 寻找中项是寻找第 k 小元素的一个特列。如果能解决寻找第 k 小元素的问题,那么当 $k=\lceil n/2 \rceil$ 时,解决的就是寻找中项问题。
- 回顾二分搜索:以中间元素为基准抛弃部分元素,不断减小问题规模。



思路

- 如果数组A中元素的个数小于一个阈值，那么采用直接的方法寻找第k小元素。
- 否则，将n个元素划分成 $\lfloor n/5 \rfloor$ 组，每组5个元素，如果n不是5的倍数，则排除剩余的元素。(注意：这并不代表第k小元素不可能位于这些排除的元素中详见后续分析)。
- 对每组元素排序，并取出它们的中项(即第3个元素)。 $\lfloor n/5 \rfloor$ 个中项的中项，我们记为mm。
- 依据mm将数组A划分为三个子数组： $A_1 = \{a | a < mm\}$ 、 $A_2 = \{a | a = mm\}$ 、 $A_3 = \{a | a > mm\}$ 。
- 判断第k小元素可能在哪一个子数组中出现：如果在 A_2 中出现，则已经找到；否在，在 A_1 或 A_3 上进行递归。



Algorithm: Select($A[\text{low} \dots \text{high}], k$)

输入：数组 $A[\text{low}, \dots, \text{high}]$ 和整数 k ,

输出： $A[\text{low} \dots \text{high}]$ 中的第 k 小元素

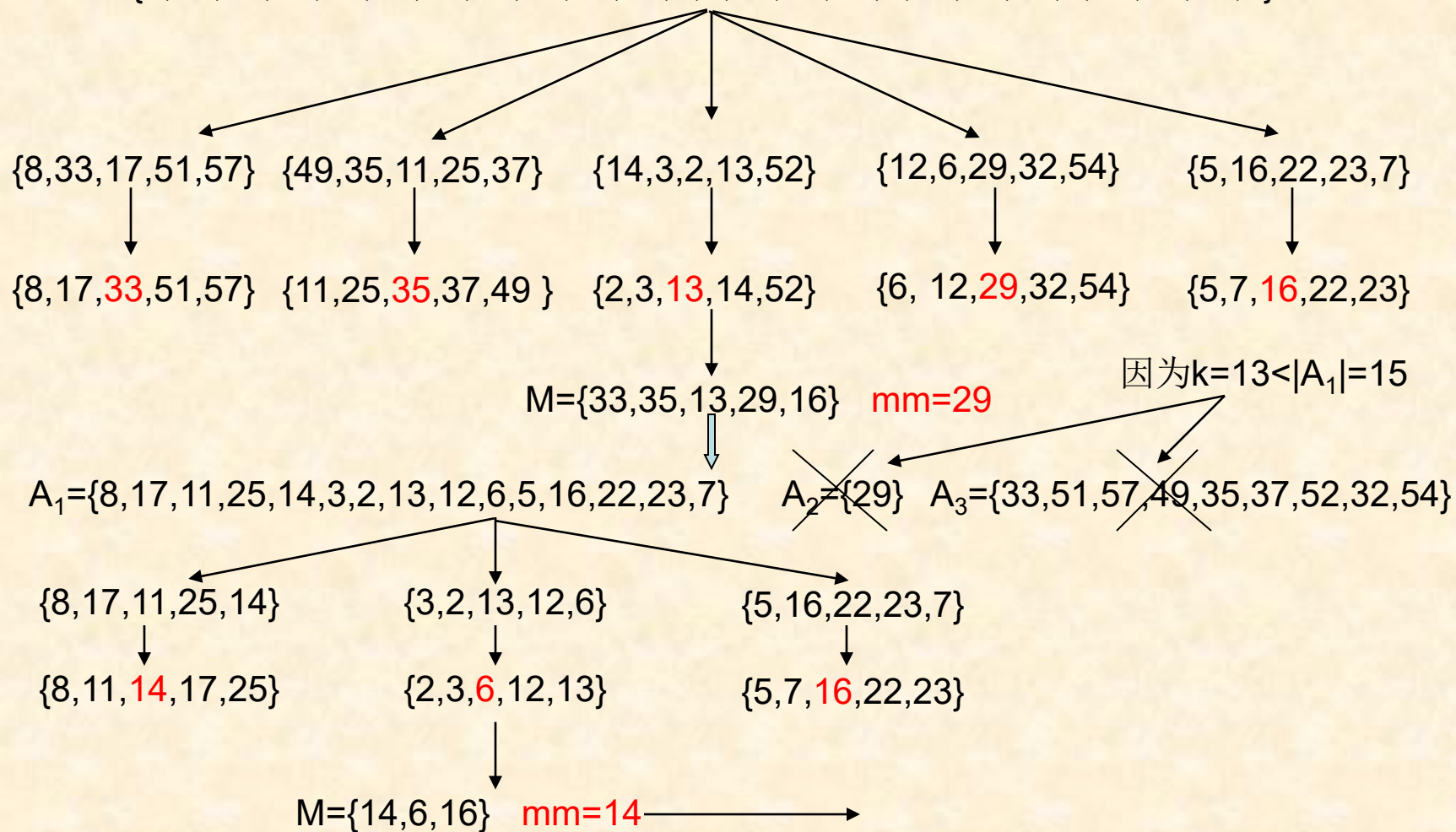
1. $p \leftarrow \text{high} - \text{low} + 1$
2. if $p < 44$ then {将 $A[\text{low}, \dots, \text{high}]$ 排序, return $A[\text{low} + k - 1]$ }
3. 令 $q = \lfloor p/5 \rfloor$ 。将 $A[\text{low} \dots \text{high}]$ 分成 q 个子数组, 每组 5 个元素。若 5 不整除 p , 则排除剩余的元素。
4. 对 q 个子数组分别进行排序, 分别找出中项。这些中项组成一个数组 M
5. $\text{mm} \leftarrow \text{Select}(M[1 \dots q], \lceil q/2 \rceil)$ // 中项的中项
6. 将 $A[\text{low} \dots \text{high}]$ 分成三组
 $A_1 = \{a | a < \text{mm}\}$ $A_2 = \{a | a = \text{mm}\}$ $A_3 = \{a | a > \text{mm}\}$ 。
7. case
| $A_1 | \geq k$: return select($A_1[1 \dots |A_1|], k$)
| $|A_1| + |A_2| \geq k$: return mm
| $|A_1| + |A_2| < k$: return select($A_3[1, |A_3|], k - |A_1| - |A_2|$)

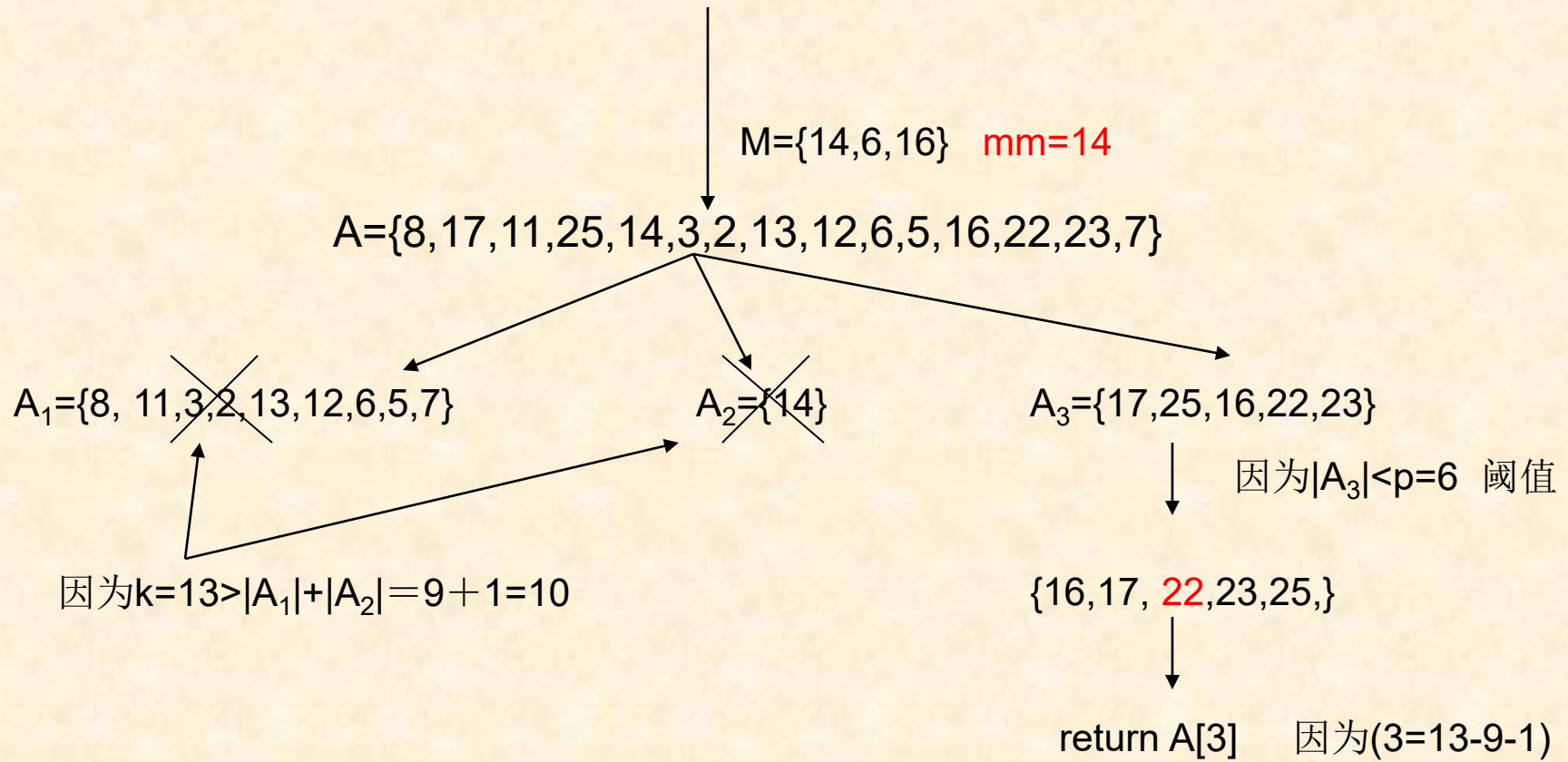


一个例子：

为方便演示，设阈值为6。现要寻找下面数组A中的第13小元素：

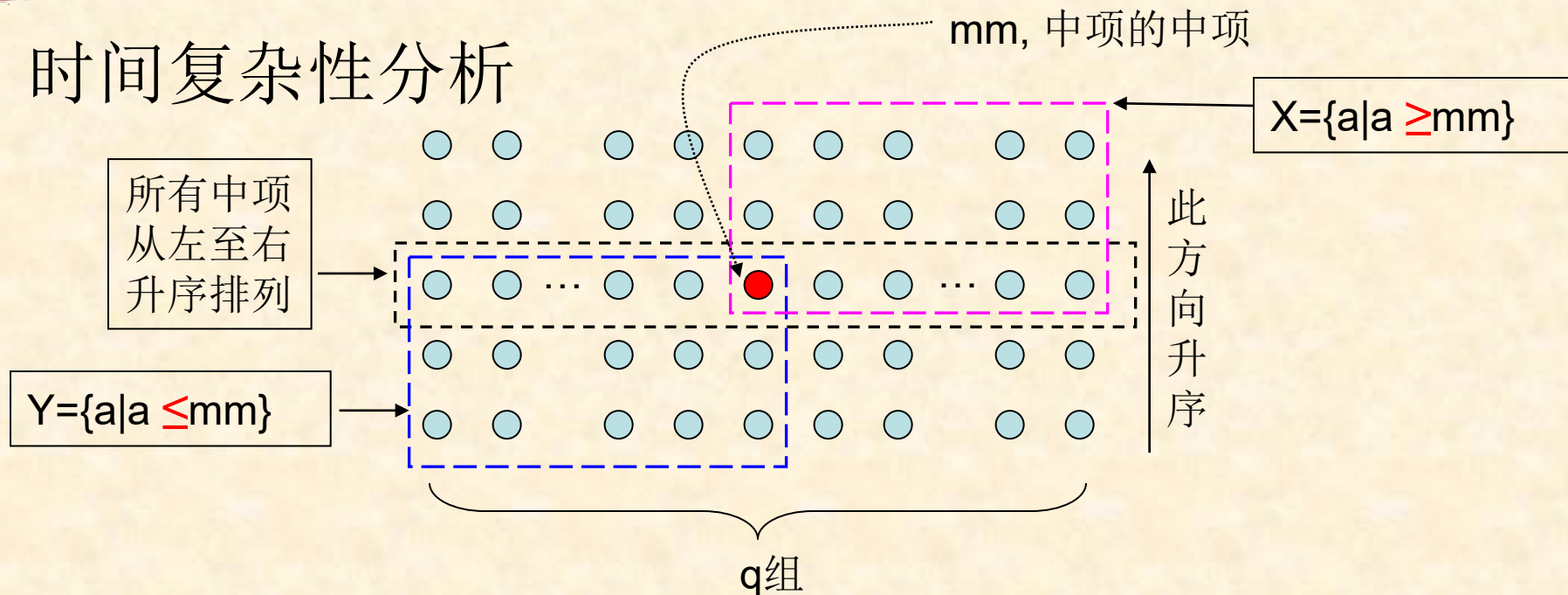
$A = \{8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 52, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7\}$







时间复杂性分析



$A1 = \{a | a < mm\}$ $A2 = \{a | a = mm\}$ $A3 = \{a | a > mm\}$

case

$|A1| \geq k$: return select($A1[1...|A1|]$, k)

$|A1| + |A2| \geq k$: return mm

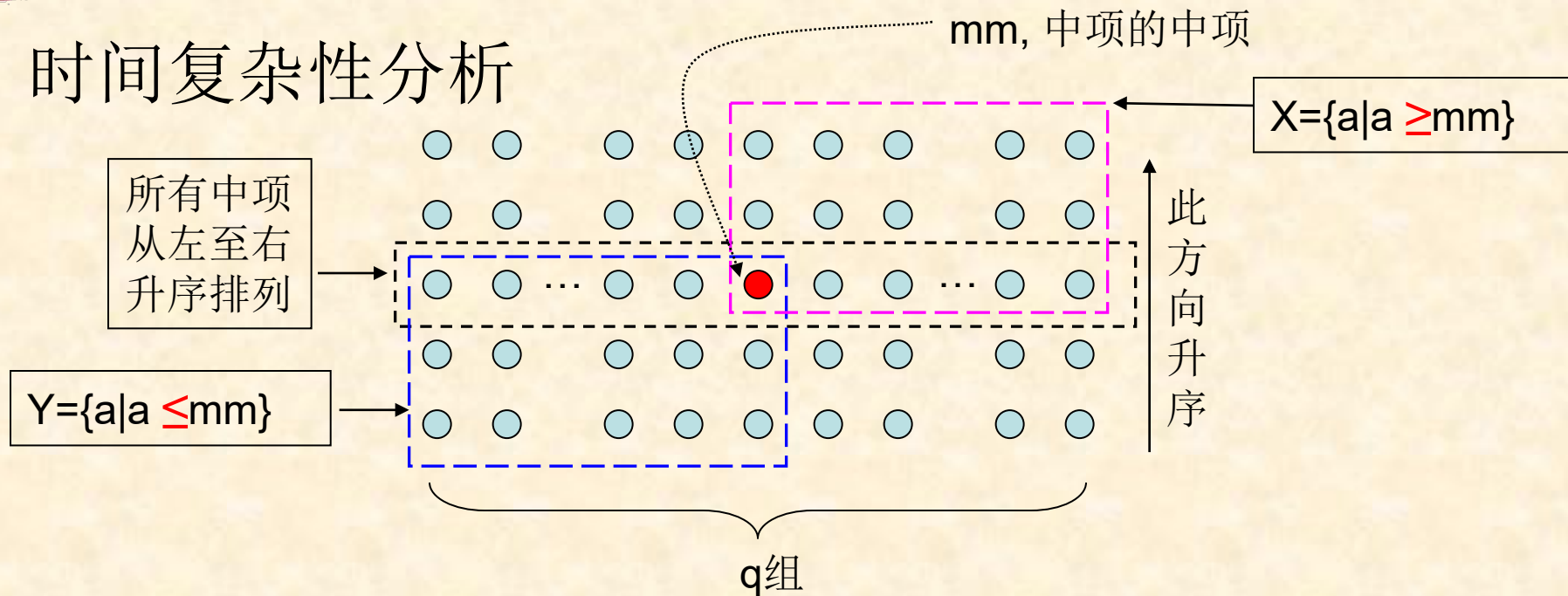
$|A1| + |A2| < k$:

return select($A3[1, |A3|]$, $k - |A1| - |A2|$)

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



时间复杂性分析



记: $A_1 = \{a | a < mm\}$, $A_1' = \{a | a \leq mm\}$,
 $A_3 = \{a | a > mm\}$, $A_3' = \{a | a \geq mm\}$,

显然有:

Y 是 A_1' 的子集,

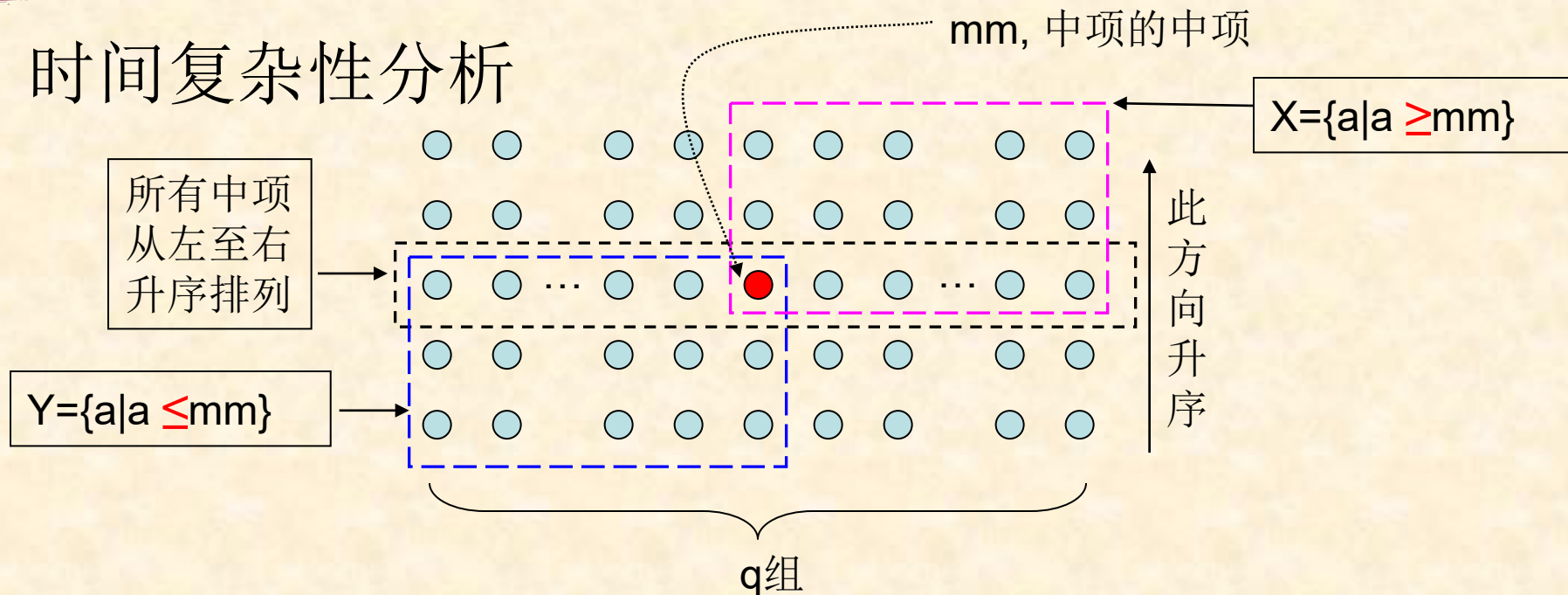
X 是 A_3' 的子集, 且

$|A_1| + |A_3| = n$, $|A_3| + |A_1'| = n$,

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



时间复杂性分析



记: $A_1 = \{a | a < mm\}$, $A_1' = \{a | a \leq mm\}$,
 $A_3 = \{a | a > mm\}$, $A_3' = \{a | a \geq mm\}$,

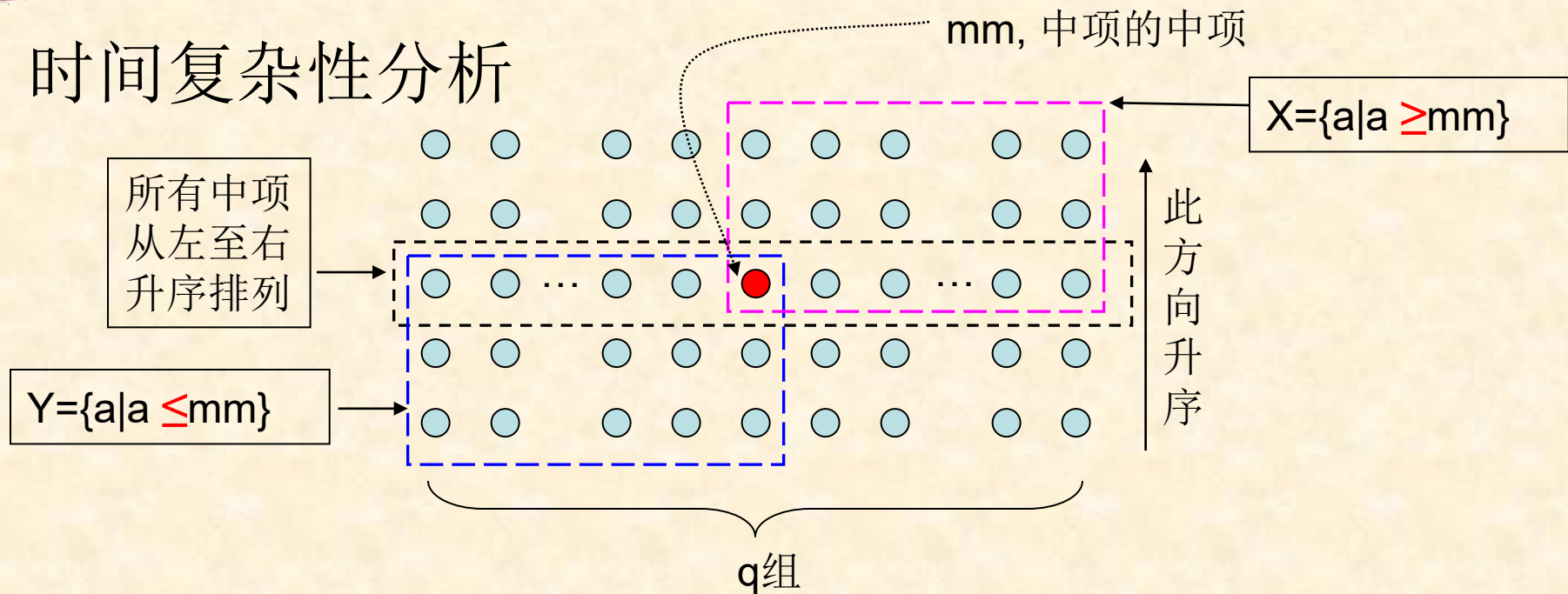
则

$$\begin{aligned}
 |A_1'| &\text{至少为 } |Y|, \text{ 即} \\
 |A_1'| &\geq |Y| = 3 * \lceil q/2 \rceil \\
 &= 3 * \lceil \lfloor n/5 \rfloor / 2 \rceil \\
 &\geq 3 * \lfloor n/5 \rfloor / 2 \\
 &= 3/2 * \lfloor n/5 \rfloor
 \end{aligned}$$

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



时间复杂性分析

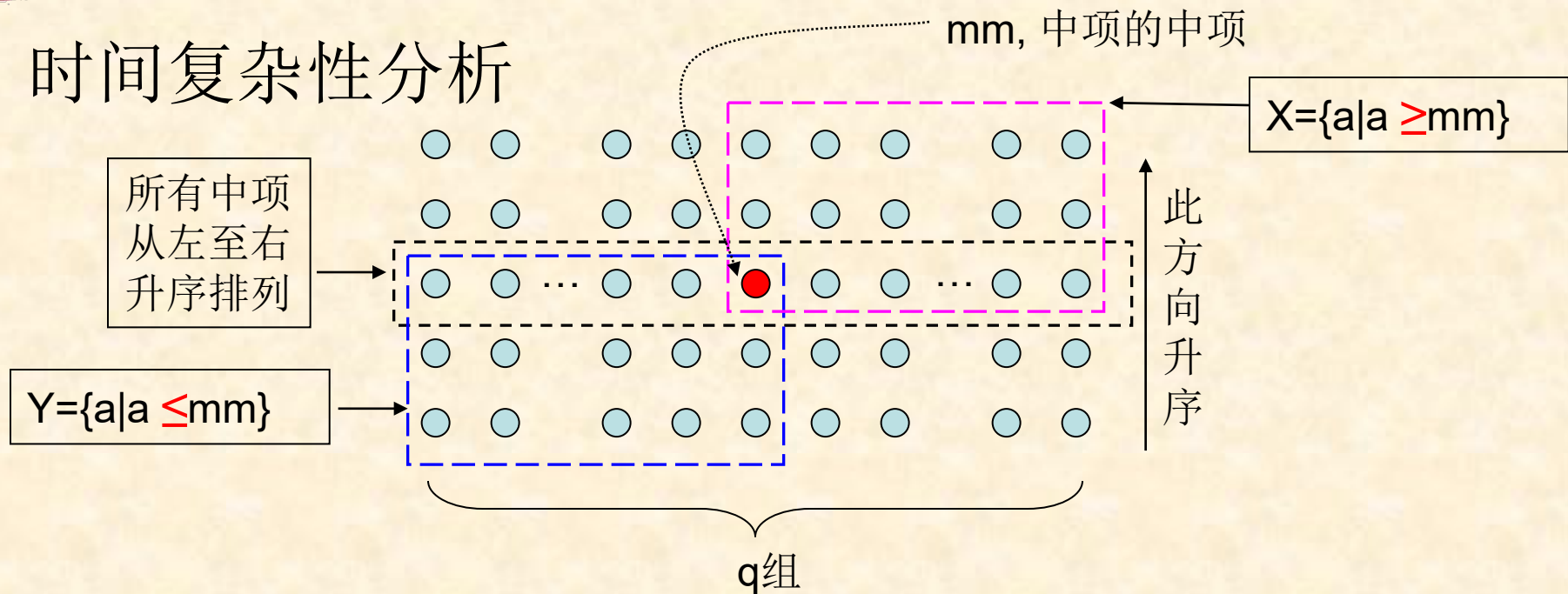


记: $A_1 = \{a | a < mm\}$, $A_1' = \{a | a \leq mm\}$,
 $A_3 = \{a | a > mm\}$, $A_3' = \{a | a \geq mm\}$,
 因 $|A_3| + |A_1'| = n$,
 $|A_1'| \geq 3/2 * \lfloor n/5 \rfloor$
 则 $|A_3| \leq n - 3/2 * \lfloor n/5 \rfloor$
 $\leq n - 3/2 * ((n-4)/5)$
 $= 0.7n + 1.2$

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



时间复杂性分析



记: $A_1 = \{a | a < mm\}$, $A_1' = \{a | a \leq mm\}$,
 $A_3 = \{a | a > mm\}$, $A_3' = \{a | a \geq mm\}$,

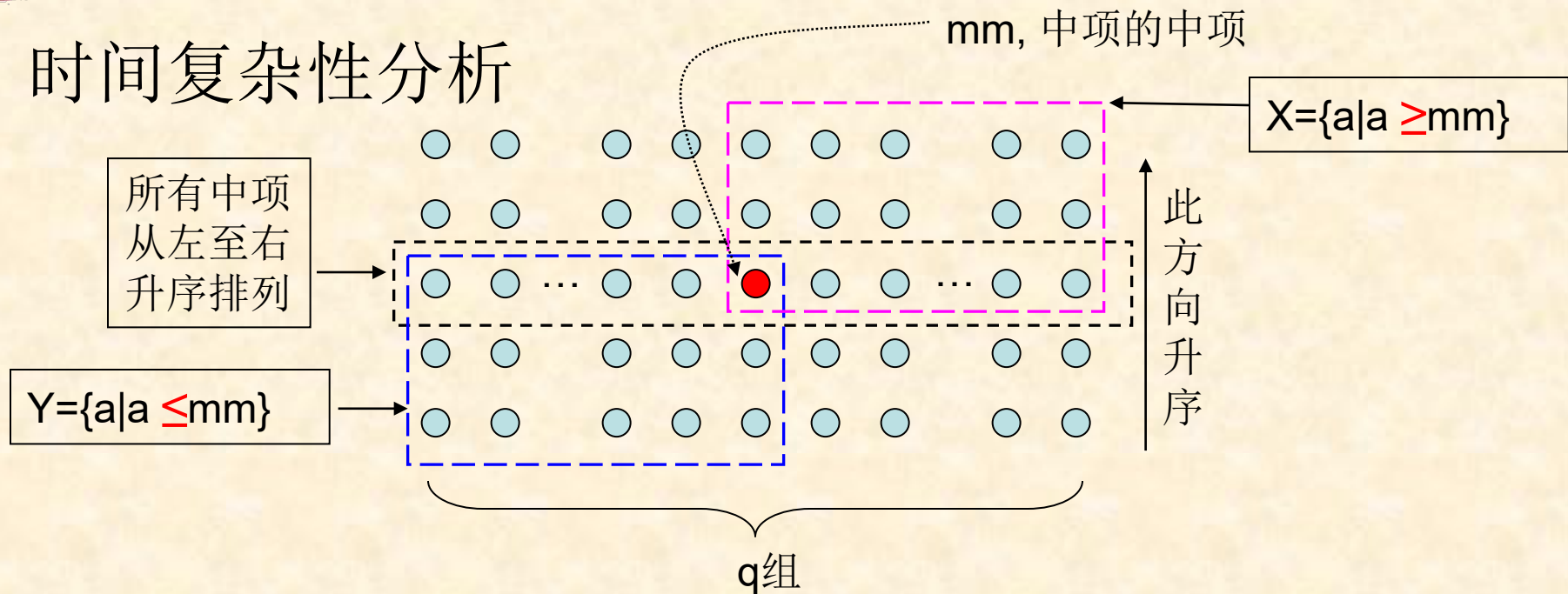
同理有:

$$\begin{aligned}
 |A_3'| &\geq |X| = 3 * \lceil q/2 \rceil \\
 &= 3 * \lceil \lfloor n/5 \rfloor / 2 \rceil \\
 &\geq 3 * \lfloor n/5 \rfloor / 2 \\
 &= 3/2 * \lfloor n/5 \rfloor
 \end{aligned}$$

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



时间复杂性分析



记: $A_1 = \{a | a < mm\}$, $A_1' = \{a | a \leq mm\}$,
 $A_3 = \{a | a > mm\}$, $A_3' = \{a | a \geq mm\}$,
 因 $|A_1| + |A_3| = n$,
 $|A_3'| \geq 3/2 * \lfloor n/5 \rfloor$
 则 $|A_1| \leq n - 3/2 * \lfloor n/5 \rfloor$
 $\leq n - 3/2 * ((n-4)/5)$
 $= 0.7n + 1.2$

52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11



52	48	54	57	49
14	22	32	51	37
13	16	29	33	35
3	7	12	17	25
2	5	6	8	11

右下图给出了一种更复杂的情况，
但不会改变刚才的结论

即：小于等于中值 mm 的元素个数，
至少有蓝色框内元素那么多。

同理：

大于等于中值 mm 的元素个数，
至少有红色框内元素那么多。

52	48	54	57	49
14	31	32	29	37
13	29	29	29	35
3	7	12	17	25
2	5	6	8	11



用 $T(n)$ 表示从 n 个元素中选择第 k 小元素所需要耗费的时间。

1. $p \leftarrow \text{high} - \text{low} + 1$ // $\Theta(1)$, 这里 $p=n$
2. if $p < 44$ then {将 $A[\text{low} \dots \text{high}]$ 排序, return $A[\text{low} + k - 1]$ // $\Theta(1)$ }
3. 令 $q = \lfloor p/5 \rfloor$ 。将 $A[\text{low} \dots \text{high}]$ 分成 q 个子数组, 每组 5 个元素。若 5 不整除 p , 则排除剩余的元素。 // $\Theta(n)$
4. 对 q 个子数组分别进行排序, 分别找出中项。这些中项组成一个数组 M // $\Theta(n)$
5. $mm \leftarrow \text{Select}(M[1 \dots q], \lceil q/2 \rceil)$ // 中项的中项 // $T(\lfloor n/5 \rfloor)$
6. 将 $A[\text{low} \dots \text{high}]$ 分成三组
 $A_1 = \{a | a < mm\}$ $A_2 = \{a | a = mm\}$ $A_3 = \{a | a > mm\}$ 。 // $\Theta(n)$
7. case // 至多为 $T(0.7n + 1.2)$
 - $|A_1| \geq k$: return select($A_1[1 \dots |A_1|]$, k)
 - $|A_1| + |A_2| \geq k$: return mm
 - $|A_1| + |A_2| < k$: return select($A_3[1, |A_3|]$, $k - |A_1| - |A_2|$)



- 至此，我们为A1和A3中的元素个数建立了一个上界：即小于mm的元素个数或是大于mm的元素的个数均不能超过 $0.7n+1.2$ 。
- $T(n)$ 表示从n个元素中选择第k小元素所需要耗费的时间。
- Step 7耗费时间至多为 $T(0.7n+1.2)$ 。下面设法去掉其中的常数1.2。

假设 $0.7n+1.2 \leq \lfloor 0.75n \rfloor$ ，因 $0.75n - 1 < \lfloor 0.75n \rfloor \leq 0.75n$ ，那么当 $0.7n+1.2 \leq 0.75n-1$ ，即当 $n \geq 44$ 时， $0.7n+1.2 \leq \lfloor 0.75n \rfloor$ 成立。{//或令 $0.7n+1.2 \leq 3n/4-3/4 \rightarrow n \geq 39$ } 此时，Step 7耗费时间之多为 $T(\lfloor 0.75n \rfloor)$ 。

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + \Theta(n) & \text{if } n \geq 44 \end{cases}$$



定理2.7

$$T(n) = \Theta(n)$$



进一步的研究

- 取每组5个元素，有

$$|A_1| \leq n - 3 * \lceil \lfloor n/5 \rfloor / 2 \rceil \leq n - 3/2 ((n-4)/5) = 0.7n + 1.2$$

- 每组5个元素排序求中值，元素比较次数约为 $5\log 5 \approx 11.6$ 次
- 或估计为 $n(n-1)/2 = 10$ 次，共计

$$\left. \frac{m(m-1)}{2} \right|_{m=5} \times \left\lfloor \frac{n}{5} \right\rfloor \approx \frac{5 \times 4}{2} \times \frac{n}{5} = 2n$$

- 令 $q = \lfloor p/5 \rfloor$ 。将 $A[\text{low} \dots \text{high}]$ 分成 q 个子数组，每组5个元素。若5不整除 p ，则排除剩余的元素。// $1*n$
- 对 q 个子数组分别进行排序，分别找出中项。这些中项组成一个数组 M // $2*n$
- 将 $A[\text{low} \dots \text{high}]$ 分成三组
 $A_1 = \{a | a < m\}$ $A_2 = \{a | a = m\}$ $A_3 = \{a | a > m\}$ 。// $1*n$

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + 4n & \text{if } n \geq 44 \end{cases} \longrightarrow T(n) \leq \frac{4n}{1 - \frac{1}{5} - \frac{3}{4}} = 80n$$



进一步的研究

- 取每组5个元素，有

$$|A_1| \leq n - 3 * \lceil \lfloor n/5 \rfloor / 2 \rceil \leq n - 3/2 ((n-4)/5) = 0.7n + 1.2$$

- 若取每组7个元素，则有

$$|A_1| \leq n - 4 * \lceil \lfloor n/7 \rfloor / 2 \rceil \leq n - 4/2 ((n-6)/7) = 5n/7 + 12/7$$

令 $5n/7 + 12/7 \leq 3n/4 - 3/4 \leq \lfloor 3n/4 \rfloor$ ，得：

当 $n \geq 69$ 时， $5n/7 + 12/7 \leq \lfloor 3n/4 \rfloor$ 成立。



进一步的研究

- 每组7个元素排序求中值，元素比较次数约为 $7\log 7 \approx 19.6$ 次或估计为 $n(n-1)/2=21$ 次，共计

$$\left. \frac{m(m-1)}{2} \right|_{m=7} \times \left\lfloor \frac{n}{7} \right\rfloor \approx \frac{7 \times 6}{2} \times \frac{n}{7} = 3n$$

3. 令 $q = \lfloor p/5 \rfloor$ 。将 $A[\text{low} \dots \text{high}]$ 分成 q 个子数组，每组5个元素。若5不整除 p ,则排除剩余的元素。// $1*n$
4. 对 q 个子数组分别进行排序，分别找出中项。这些中项组成一个数组 M // $3*n$
6. 将 $A[\text{low} \dots \text{high}]$ 分成三组
 $A1 = \{a | a < mm\}$ $A2 = \{a | a = mm\}$ $A3 = \{a | a > mm\}$ 。// $1*n$

$$T(n) \leq \begin{cases} c & \text{if } n < 69 \\ T(\lfloor n/7 \rfloor) + T(\lfloor 3n/4 \rfloor) + 5n & \text{if } n \geq 69 \end{cases} \longrightarrow T(n) \leq \frac{5n}{1 - \frac{1}{7} - \frac{3}{4}} \approx 46.7n < 80n$$



进一步的研究

- 寻找中值mm比较费力，为什么要用中值mm（而不是以均值）作为基准元素？

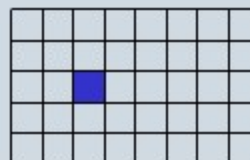
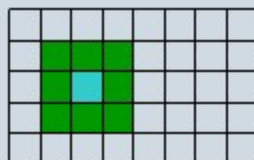


进一步的研究

均值滤波器

—— 原理

- 在图像上，对**待处理的像素**给定一个模板，该模板包括了其周围的邻近像素。将模板中的全体像素的**均值**来替代原来的像素值的方法。



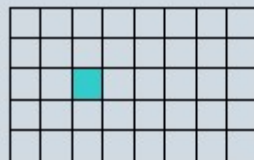
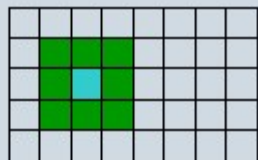


进一步的研究

中值滤波器

—— 滤波处理方法

- 与均值滤波类似，做3*3的模板，对9个数排序，取第5个数替代原来的像素值。





进一步的研究

中值滤波器与均值滤波器的比较

- 原因：
- 椒盐噪声是幅值近似相等但随机分布在不同位置上，图像中有干净点也有污染点。
- 中值滤波是选择适当的点来替代污染点的值，所以处理效果好。
- 因为噪声的均值不为0，所以均值滤波不能很好地去除噪声点。



进一步的研究



含有椒盐噪声的
Lena图像



进一步的研究



中值滤波几乎完全
去除了椒盐噪声



进一步的研究



均值滤波使椒盐噪声变成小气泡，同时图像变模糊



最接近点对问题

- 给定平面上的点集 S ， $|S|=n$ 。
- 若 $p \in S, q \in S, p \neq q$, 则 (p, q) 称为一个点对。
- $d(p, q)$ 表示该点对 (p, q) 中点 p 和点 q 之间的欧几里得距离，
- 问 $d(p, q)$ 的最小值是多少？

$$\delta = \min_{p \in S, q \in S, p \neq q} d(p, q)$$

- 严格说来，最接近点对可能多于1，为了简单起见，我们找到其中的1对作为问题的解。
- 最直观的方法：将每一点与其它 $n-1$ 个点的距离算出来，然后找出其中最小的即可。 $T(n) = \Theta(n(n-1)/2) = \Theta(n^2)$

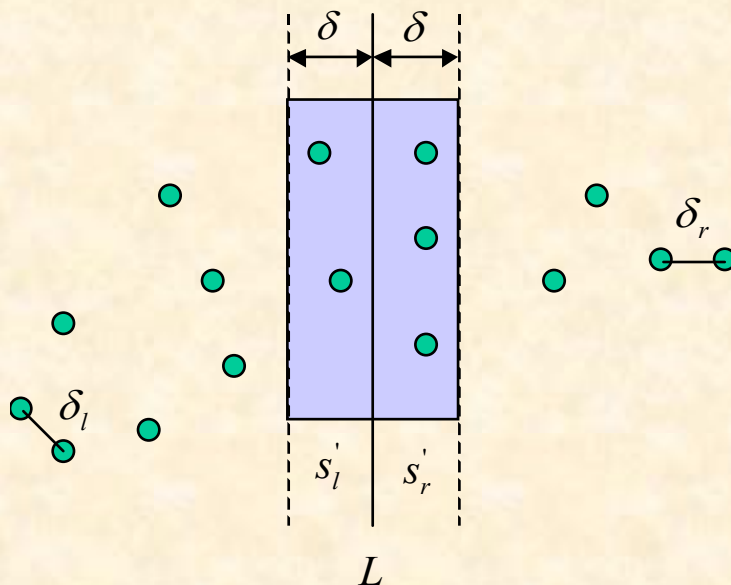


使用分治策略

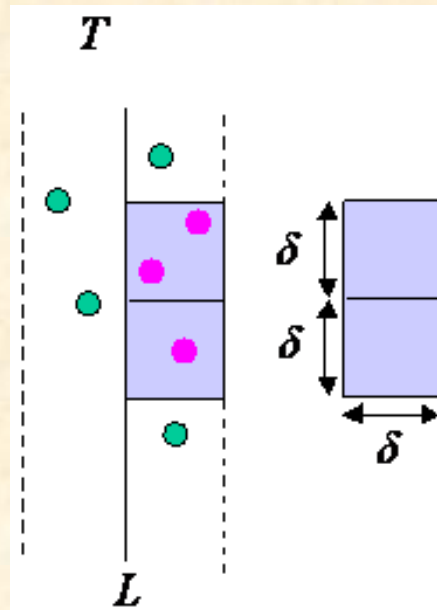
- 首先，将点集 S 中的点按照 x 坐标进行排序。
- 其次，使用一根垂直于 x 轴的直线 L 将 S 分割为两个子集 S_l 和 S_r ，使得 $|S_l| = \lfloor |S|/2 \rfloor$ ， $|S_r| = \lceil |S|/2 \rceil$ 。 S_l 中的点都落在直线 L 的左边或是 L 上； S_r 中的点都落在直线 L 的右边或是 L 上。
- 计算 S_l 中的最小距离 δ_l ， S_r 中的最小距离 δ_r (递归)； 计算 S_l 中的点与 S_r 中的点之间的最短距离 δ' (难点所在)。
- 最接近点对的距离为 $\min\{\delta_l, \delta_r, \delta'\}$ 。



计算 S_l 中的点与 S_r 中的点之间的最短距离 δ' (难点所在)



- 直接的方法：求解 S_l 中每个点与 S_r 中每个点之间距离，然后找出最小值。然而，这种方法需要的时间 $T(n)=\Theta(n^2)$ 。
- 我们设 $\delta=\min\{\delta_l, \delta_r\}$ ，如果最近邻点对是由 S_l 中的某个点 p 和 S_r 中的某个点 q 所组成，那么， p, q 至 L 的水平距离均不会超过 δ 。
- 用 S'_l 表示 S_l 中与 L 水平距离小于等于 δ 的点集，用 S'_r 表示 S_r 中与 L 水平距离小于等于 δ 的点集。也就是有 $p \in S'_l, q \in S'_r$
- 然而，计算 S'_l 和 S'_r 中点对的最小值在最坏情形下时间复杂性仍旧为 $T(n)=\Theta(n^2)$ （当 $S'_l = S_l$ 和 $S'_r = S_r$ 时）。

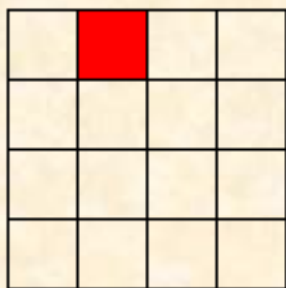


- 用 T 表示两个垂直带之间的点的集合。
- 进一步分析，可以发现：如果最近邻点对是由 S_l 中的某个点 p 和 S_r 中的某个点 q 所组成，那么， p, q 之间的垂直距离不会超过 δ 。
- 在一个高为 2δ , 宽为 δ 的矩形内，任意两点之间的距离不超过 δ ，那么该矩形内可容纳不超过 6 个点(为什么？)。所以 T 中的每个点至多只需要和 T 中的 5 个点计算距离($O(5n)$)，而不需要计算和 T 中所有的其它点之间的距离($O(n^2)$)。
- 至此，问题得解。课后自行阅读 CLOSESTPAIR 算法，及时间复杂性分析。



棋盘覆盖问题

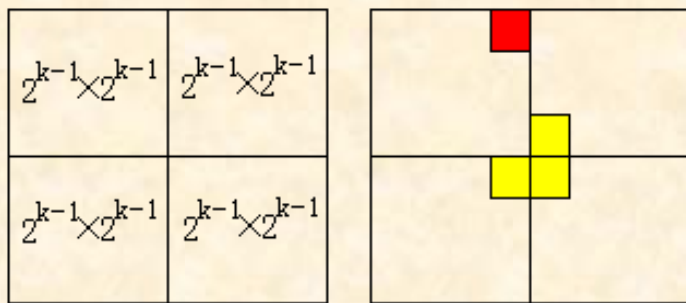
在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其他方格不同，称该方格为一**特殊方格**(红色表示)，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。





分析

当 $k > 0$ 时，可以将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示。特殊方格**必位于4个较小子棋盘之一中**，其余3个子棋盘中无特殊方格。为了将这3个无特殊方格的子棋盘转化为特殊棋盘，可以用一个L型骨牌覆盖这3个较小棋盘的会合处，如(b)所示，从而将原问题转化为4个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为棋盘 2×2 。



(a)

(b)





棋盘覆盖问题实例

2	2	3	3	7	7	8	8
2	1	1	3	7	6	6	8
4	1	5	5	9	9	6	10
4	4	5	0		9	10	10
12	12	13	0	0	17	18	18
12	11	13	13	17	17	16	18
14	11	11	15	19	16	16	20
14	14	15	15	19	19	20	20

一个实例



算法描述

```
public void chessBoard(int tr, int tc, int dr, int dc, int size)
{
    if (size == 1) return;
    int t = tile++, // 全局整型变量，表示L型骨牌编号
    s = size/2; // 分割棋盘
    // 覆盖左上角子棋盘
    if (dr < tr + s && dc < tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr, tc, dr, dc, s);
    else
    { // 此棋盘中无特殊方格
        // 用 t 号L型骨牌覆盖右下角
        board[tr + s - 1][tc + s - 1] = t;
        // 覆盖其余方格
        chessBoard(tr, tc, tr+s-1, tc+s-1, s);
    }

    // 覆盖右上角子棋盘
    if (dr < tr + s && dc >= tc + s)
        // 特殊方格在此棋盘中
        chessBoard(tr, tc+s, dr, dc, s);
    else // 此棋盘中无特殊方格
    { // 用 t 号L型骨牌覆盖左下角
        board[tr + s - 1][tc + s] = t;
        // 覆盖其余方格
        chessBoard(tr, tc+s, tr+s-1, tc+s, s);
    }
}
```

```
// 覆盖左下角子棋盘
if (dr >= tr + s && dc < tc + s)
    // 特殊方格在此棋盘中
    chessBoard(tr+s, tc, dr, dc, s);
else
    { // 用 t 号L型骨牌覆盖右上角
        board[tr + s][tc + s - 1] = t;
        // 覆盖其余方格
        chessBoard(tr+s, tc, tr+s, tc+s-1, s);
    }

// 覆盖右下角子棋盘
if (dr >= tr + s && dc >= tc + s)
    // 特殊方格在此棋盘中
    chessBoard(tr+s, tc+s, dr, dc, s);
else
    { // 用 t 号L型骨牌覆盖左上角
        board[tr + s][tc + s] = t;
        // 覆盖其余方格
        chessBoard(tr+s, tc+s, tr+s, tc+s, s);
    }
}
```

Board[0][0]: 原棋盘左上角的方格

tr: 棋盘左上角方格的行号

tc: 棋盘左上角方格的列号 $T(k) = 4T(k-1) + O(1)$

dr: 特殊方格所在的行号

dc: 特殊方格所在的列号

size: 棋盘大小

$$= \Theta(4^k)$$



循环赛日程表问题

设有 $n=2^k$ 个运动员进行网球循环赛，比赛日程设计要求：

- 1) 每个选手必须与其余 $n-1$ 个选手各赛一次
- 2) 每个选手每天只能赛一次
- 3) 循环赛一共进行 $n-1$ 天

设计循环赛日程表，表的第 i 行第 j 列填入第 i 个选手在第 j 天所遇到的选手。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

图 2-12 8 个选手的比赛日程表



算法代码

```
Void Table(int k, int *a)
{
    int n = 1;
    for (int i=1;i<=k;i++) n*=2;
    for (int i=1;i<=n;i++) a[1][i]=i;
    int m=1;
    for (int s=1;s<=k;s++){
        n/=2;
        for (int t=1; t<=n; t++)
            for (int i=m+1;i<=2*m;i++)
                for (int j=m+1;j<=2*m;j++){
                    a[i][j+(t-1)*m*2] = a[i-m][j+(t-1)*m*2-m];
                    a[i][j+(t-1)*m*2-m] = a[i-m][j+(t-1)*m*2];
                }
            m*=2;
        }
    }
```



小结

- 递归

- 优点：结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。
- 缺点：递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

- 分治法的适用条件

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的同类问题；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划更为合适。



作业

- **A: 6.6, 6.16, 6.38**
- **B: 实验作业（可选）：**

实现线性查找第 k 小元素的算法，以表格形式给出若干组 $(n, T(n))$ 数据，其中， n 是问题规模， $T(n)$ 是元素比较次数；并绘图展示结果。